

OPERATING SYSTEMS (UE24CS242B)

Unit 1 – CPU Scheduling Algorithms – Priority & Round Robin

Priority Scheduling

In priority scheduling, a priority number (integer) is associated with each process.

Scheduling Rule

- CPU is allocated to the process with the highest priority
- Convention used: smaller integer $\Rightarrow$ higher priority

Types

- **Preemptive priority scheduling**
 - A running process can be preempted if a higher-priority process arrives
- **Non-preemptive priority scheduling**
 - Running process continues until it finishes or blocks

Relation to SJF

- SJF is a special case of priority scheduling
- Priority = inverse of the predicted next CPU burst time

Problem in Priority Scheduling – Starvation

Starvation

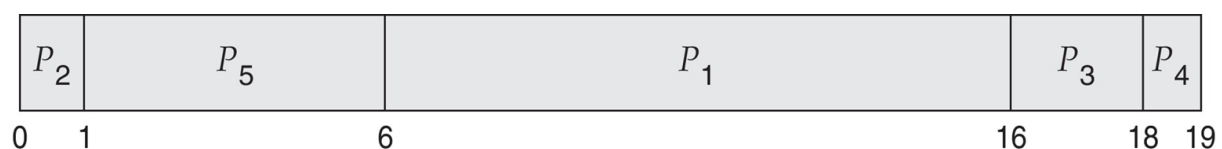
- Low-priority processes may never get CPU time
- Occurs when high-priority processes continuously arrive

Solution – Aging

- Gradually increase the priority of a process as it waits
- Ensures that every process eventually executes

Example of Priority Scheduling

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2



Average Waiting Time

$$(6 + 0 + 16 + 18 + 1)/5 = 41/5 = 8.2$$

Round-Robin (RR) Scheduling

Round-Robin scheduling is designed specifically for time-sharing systems.

Key Characteristics

- Similar to FCFS, but preemption is added
- Uses a fixed time unit called time quantum or time slice
- Typical quantum length: 10–100 milliseconds
- Ready queue is treated as a circular queue

Working

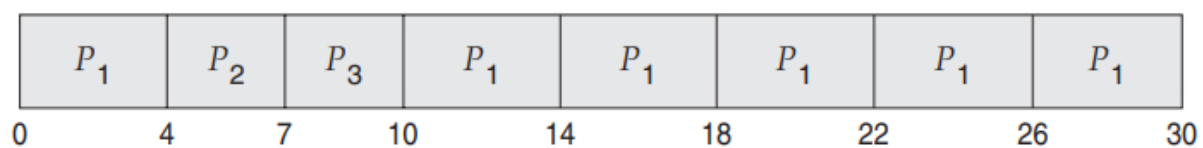
- Each process gets the CPU for at most one time quantum
- If the process does not finish within the quantum:
 - It is preempted
 - Placed at the end of the ready queue

Round-Robin Scheduling – Example

Given processes (arrival time = 0):

Process	Burst Time
P_1	24
P_2	3
P_3	3

Time quantum = **4 ms**



Waiting Time

- P_1 waits for **6 ms** ($10 - 4$)
- P_2 waits for **4 ms**
- P_3 waits for **7 ms**

Average Waiting Time

$$(6 + 4 + 7)/3 = 5.66 \text{ ms}$$

Round-Robin Scheduling – Performance

If:

- n = number of processes in ready queue
- q = time quantum

Then:

- Each process gets $1/n$ of CPU time
- CPU is allocated in chunks of at most q time units
- A process waits **no longer than** $(n - 1) \times q$ time units for its next turn

Example

- 5 processes
- Time quantum = 20 ms
→ Each process gets **20 ms every 100 ms**

Effect of Time Quantum Size

Very Large Time Quantum

- RR behaves like **FCFS**
- No effective preemption

Very Small Time Quantum

- Too many **context switches**
- High overhead
- Reduced CPU efficiency

Thus, RR performance **depends heavily on the time quantum size**.

Example – Effect of Time Quantum

Consider one process with burst time = **10 units**

- Quantum = **12**
 - Process completes in one quantum
 - No context switch overhead
- Quantum = **6**
 - Process needs 2 quanta
 - One context switch
- Quantum = **1**
 - Process needs 10 quanta
 - **9 context switches**
 - Execution slows significantly

Practical Considerations

- Modern systems typically use:
 - Time quantum = **10–100 ms**
- Context switch time:
 - Usually < **10 microseconds**
- Hence, context-switch overhead is a **small fraction** of the quantum

Turnaround Time vs Time Quantum

- Turnaround time varies with time quantum
- General guideline:
 - **80% of CPU bursts should be shorter than the time quantum**

This balances:

- Responsiveness
- Context-switch overhead
- CPU utilization